Esercitazione2Plus - Hello World, Main Args

```
class Program
{
    0 references
    static void Main(string[] args)
    {
        Console.WriteLine("Hello, World!");
        if(args.Length > 0)
        /* Controllo se ci sono argomenti da linea di comando */
        {
            Console.WriteLine("Arguments provided:");
            foreach (var arg in args)
            {
                Console.WriteLine(arg);
            }
            // TODO Restituire il 2' argomento
        }
        else
        {
            Console.WriteLine("No arguments provided.");
        }
    }
}
```

Il parametro string[] args del metodo Main permette di ricevere argomenti da riga di comando:

tutto ciò che viene scritto dopo -- nel comando dotnet run viene passato al programma come input, utile per personalizzare l'esecuzione.

Esercitazione3 - Dependency

- Nel terminale di Visual Studio Code spostarsi nella directory in cui si vuole creare il nuovo progetto;
- Creare un nuovo progetto .NET 8 di tipo console denominato Esercitazione3
  - `dotnet new console -n Esercitazione3`
- Aggiungere al progetto Esercitazione3 una dipendenza (es. Humanizer)
  - `dotnet add package Humanizer`
- Che cos'è Humanizer?

Esercitazione3 - Dependency

```xml
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Humanizer" Version="2.14.1" />
  </ItemGroup>

</Project>
```

Esercitazione3.csproj
Nel file csproj è presente un nuovo tag xml (<ItemGroup>), che contiene il riferimento al package 'Humanizer' e la sua versione.
I riferimenti ai pacchetti, tramite <PackageReference>, specificano le dipendenze dei pacchetti direttamente all'interno dei file di progetto.
All'interno della cartella bin sono presenti le dll relative a Humanizer.

Esercitazione3 - Dependency

Humanizer è un package per la manipolazione e la visualizzazione di stringhe, enum, date, orari, intervalli di tempo, numeri e quantità.

Firma del metodo Pluralize:

string string.Pluralize([bool inputIsKnownToBeSingular = true])

E’ un metodo che restituisce il plurale della stringa su cui è applicato.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione3 - Dependency

```
using Humanizer;

Console.WriteLine("Woman".Pluralize());
```

Program.cs

- Modificare il Program.cs e inserire il codice come in figura sopra che richiama il metodo Pluralize() e stampa a console il risultato restituito dal metodo;
- Eseguire l'applicazione.

A console viene visualizzata la scritta: *Women*

Esercitazione4 – Tipi di dato e classi

Tipi di dato

I tipi valore e i tipi riferimento sono le due categorie principali di tipi C#.

- Una variabile di un tipo valore contiene un'istanza del tipo. Per impostazione predefinita, per l'assegnazione, il passaggio di un argomento a un metodo e la restituzione di un risultato del metodo vengono copiati i valori delle variabili.
- Le variabili dei tipi di riferimento archiviano i riferimenti ai relativi dati. Con i tipi di riferimento, due variabili possono fare riferimento allo stesso oggetto. Di conseguenza le operazioni su una variabile possono influire sull'oggetto a cui fa riferimento l'altra variabile.

Esercitazione4 – Tipi di dato e classi

Ogni parola chiave di tipo C# è un alias per il tipo .NET corrispondente.

Alcuni Tipi valore predefiniti in C#

Parola chiave del tipo C#	Tipo .NET
bool	System.Boolean
byte	System.Byte
sbyte	System.SByte
char	System.Char
decimal	System.Decimal
double	System.Double
float	System.Single
int	System.Int32
uint	System.UInt32
long	System.Int64
ulong	System.UInt64
short	System.Int16
ushort	System.UInt16

Alcuni Tipi di riferimento predefiniti in C#

Parola chiave del tipo C#	Tipo .NET
object	System.Object
string	System.String

Alcuni Valori predefiniti dei tipi in C#

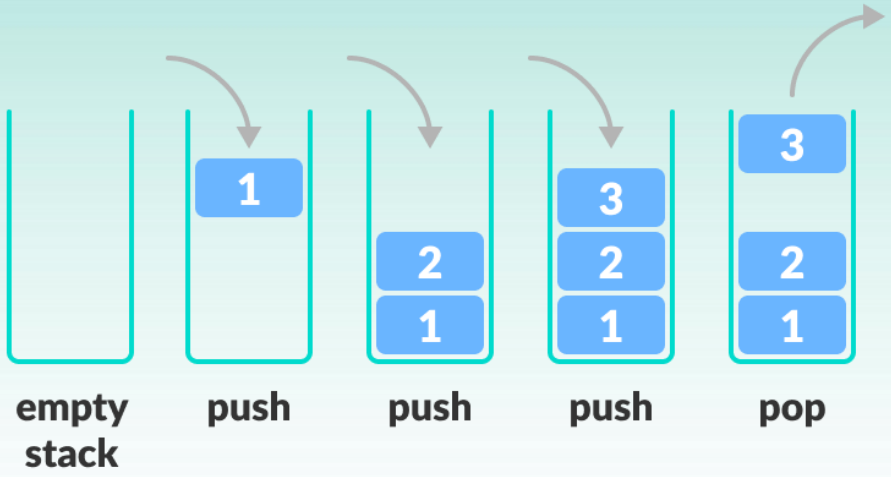
Type	Default value
Qualsiasi tipo riferimento	null
Qualsiasi tipo numerico	0 (zero)
bool	false

Esercitazione4 – Tipi di dato e classi

Stack e Heap

Nei value e reference types in C Sharp quando un programma C# va in esecuzione l'ambiente di runtime crea due aree di memoria ben distinte, lo stack e lo heap. Iniziamo a parlare dello stack.

Lo stack (pila in italiano) è un'area di memoria molto veloce, una struttura di tipo LIFO (Last In First Out) cioè l'ultimo valore immagazzinato è il primo ad uscire.



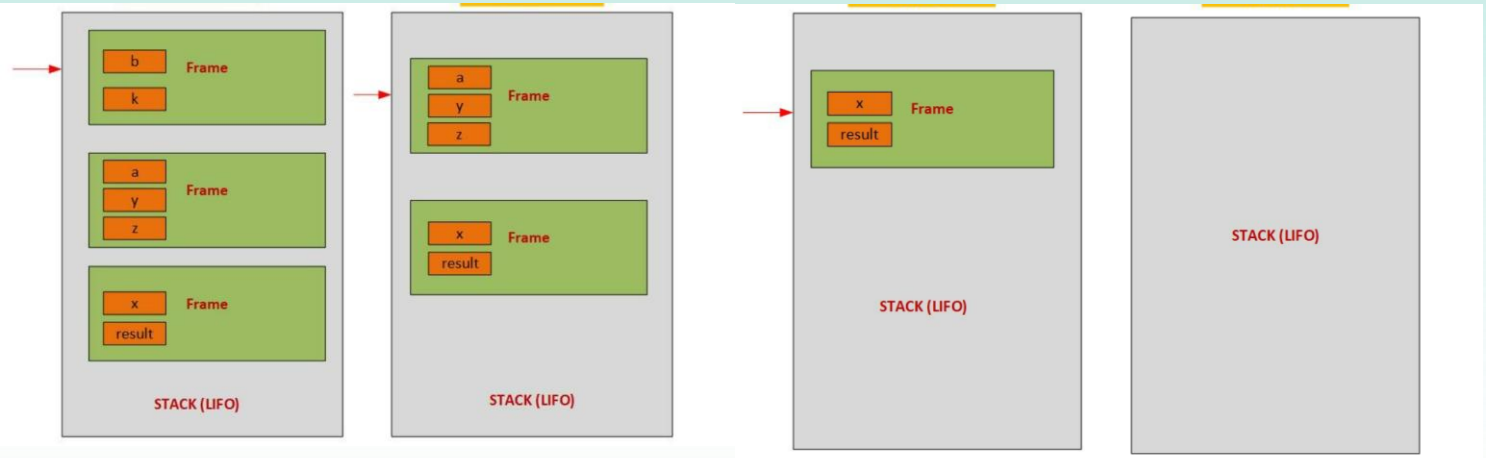
COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione4 – Tipi di dato e classi

Stack e Heap

Lo stack tiene traccia del flusso di esecuzione del programma. Abbiamo visto che in una applicazione console il metodo Main è il primo ad essere eseguito. Quando il programma entra in questo metodo nello stack viene creato un cosiddetto frame che memorizza tutti i parametri del metodo.

Quando arriva la call al FirstMethod l'esecuzione del programma salta a questo metodo. Sullo stack viene creato un nuovo frame in cui vengono immagazzinati il parametro di input a e le due variabili locali y e z. Il frame appena creato occupa la cima dello stack Figura 2. Quando viene chiamato SecondMethod viene creato nuovamente un frame che memorizza tutti parametri e variabili di SecondMethod Figura 3 e l'esecuzione prosegue con l'esecuzione di quest'ultimo. Al termine di SecondMethod si risale la cima dello stack in particolare tutte i parametri e le variabili di questo metodo vengono cancellati dalla memoria e si passa di nuovo ad eseguire FirstMethod e così via fino all'ultima istruzione del metodo Main che termina il programma (Figura 4).



COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

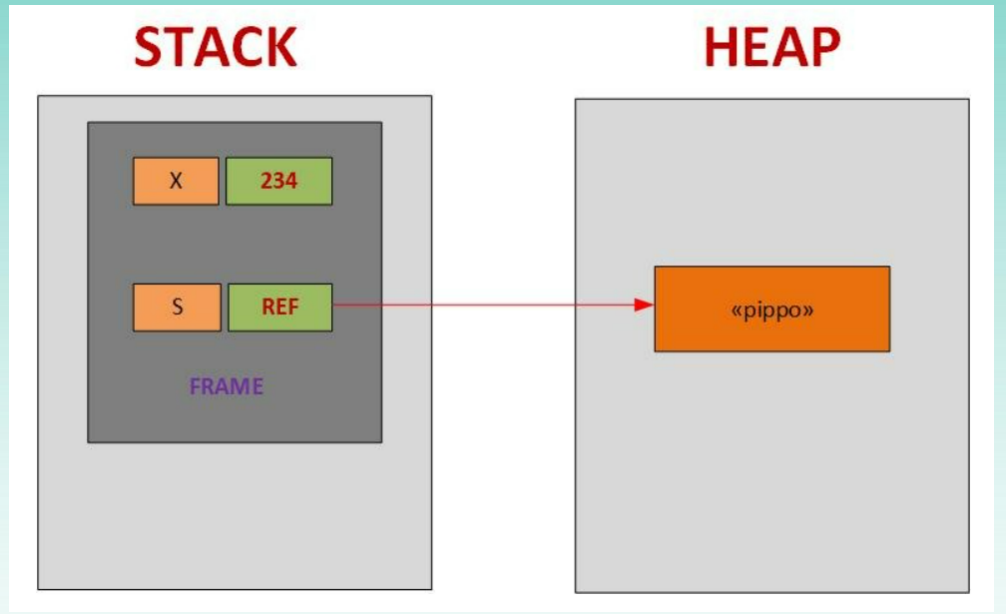
Esercitazione4 – Tipi di dato e classi

Stack e Heap

L'Heap (catasta mucchio in italiano) a differenza dello stack è un'area di memoria in cui non viene tracciata l'esecuzione del programma, questo è compito dello stack ma è un'area di memoria disordinata in cui vengono memorizzati gli oggetti per un certo tempo. È compito del Garbage Collector quando non ci sono più riferimenti ad un oggetto liberare la memoria, ma questo lo vedremo meglio parlando dei Value Types e Reference Types.

Vediamo con una figura come l'ambiente di runtime gestisce il tipo int (Value Types) e il tipo string (Reference Types).

Per quanto riguarda il comportamento dei value types come ad esempio un int vediamo che nello stack oltre alla variabile x viene memorizzato direttamente il valore 234 ossia la variabile x contiene o per meglio dire punta direttamente il valore 234. Per quanto riguarda invece le stringhe che abbiamo già incontrato, nello stack viene memorizzata la variabile e il riferimento alla stringa ossia nel reference è contenuta la locazione di memoria che si trova nello heap e che contiene il valore "pippo". Questa è la grossa differenza tra un value types che memorizzano direttamente il valore insieme alla variabile nello stack e i reference types che nello stack o meglio nel frame creato dal runtime memorizzano una variabile che punta il riferimento all'oggetto memorizzato sullo Heap.



Esercitazione4 – Tipi di dato e classi

Supponiamo che il nostro programma esegua il metodo Main nel quale abbiamo dichiarato due variabili x e y di tipo intero e assegniamo il valore di x ad y. Il runtime come sai crea un frame nello stack e all'interno del frame memorizza le variabili x e y insieme ai loro valori.

int x = 120;

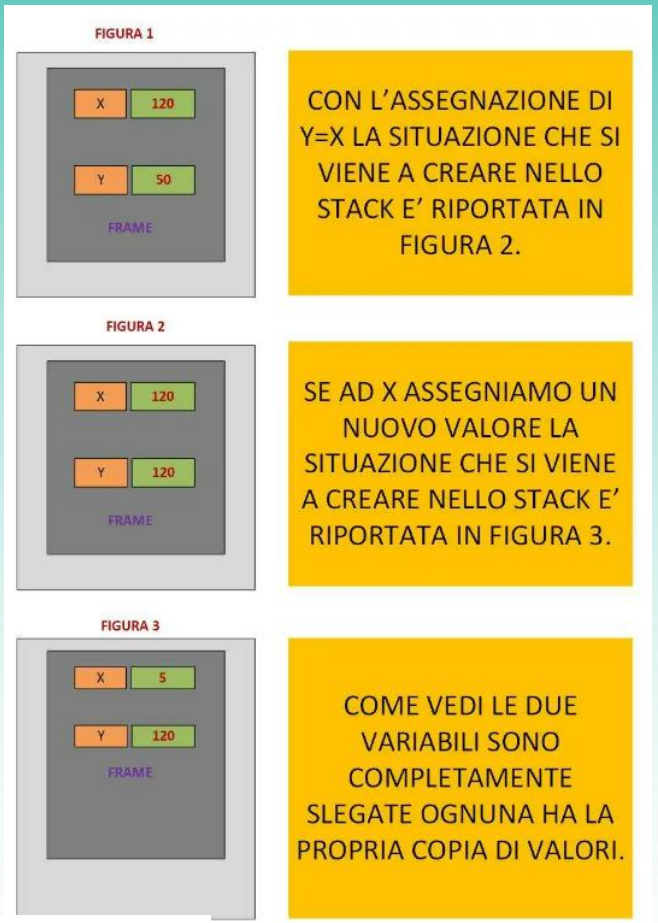
int y = 50;

y=x;

x=5;

vediamo con una figura la situazione che si crea nello stack.

Quando il flusso del programma nel Main assegna il valore di x ad y vengono create due copie separate per il valore literal 120 una per x e l'altra per y. Quando riassegniamo il valore ad x (x=5) si vede che y mantiene il precedente valore e x assume 5. Ciò sta a dimostrare che per i value types le due variabili sono completamente slegate.



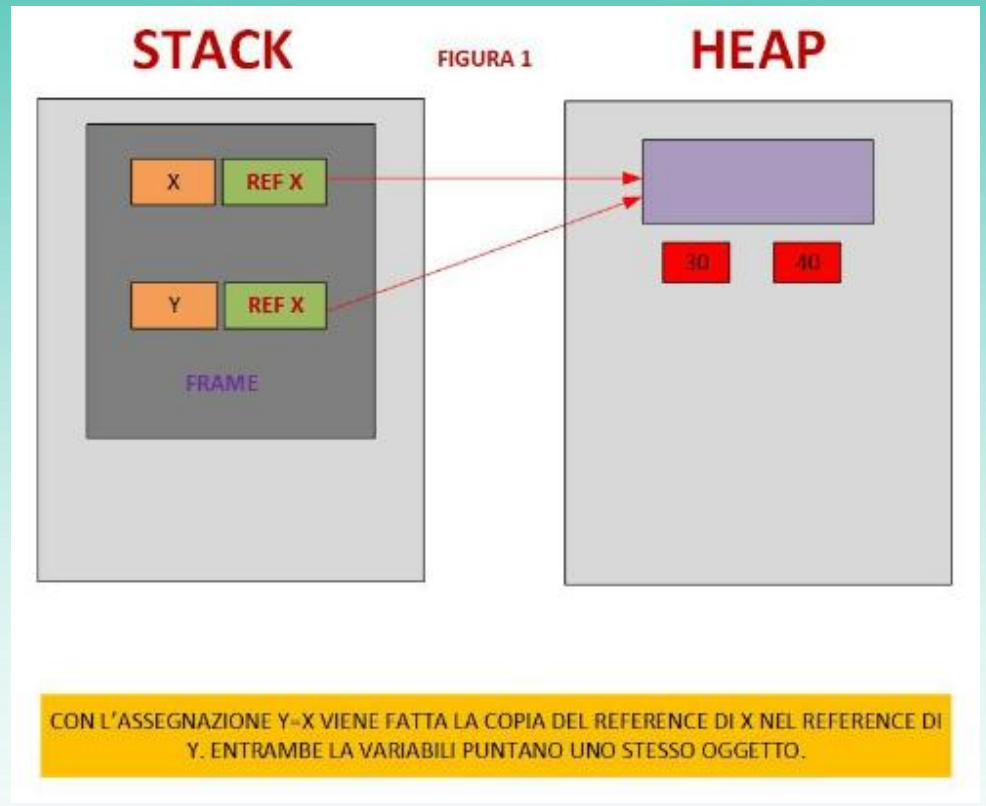
Esercitazione4 – Tipi di dato e classi

Supponiamo che nel nostro metodo Main dichiariamo un array di int, gli array li vedremo successivamente. Un array è una collezione di oggetti di un certo tipo la sintassi per dichiarare un array è riportata più in basso. Poi supponiamo di creare un secondo array y ed assegnarlo a x; Infine modifichiamo il primo elemento di x. Vediamo con una figura come si presenta la situazione in memoria.

int [] x = **new** [] **{30,40};**

int [] **y = x;**

x[0] = 10;

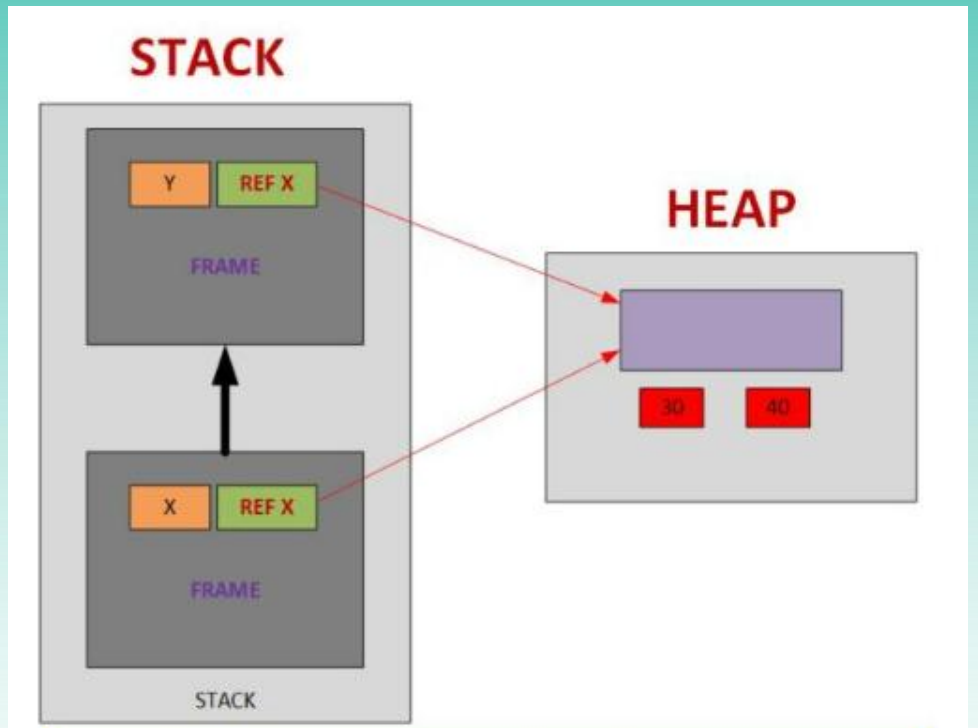


COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione4 – Tipi di dato e classi



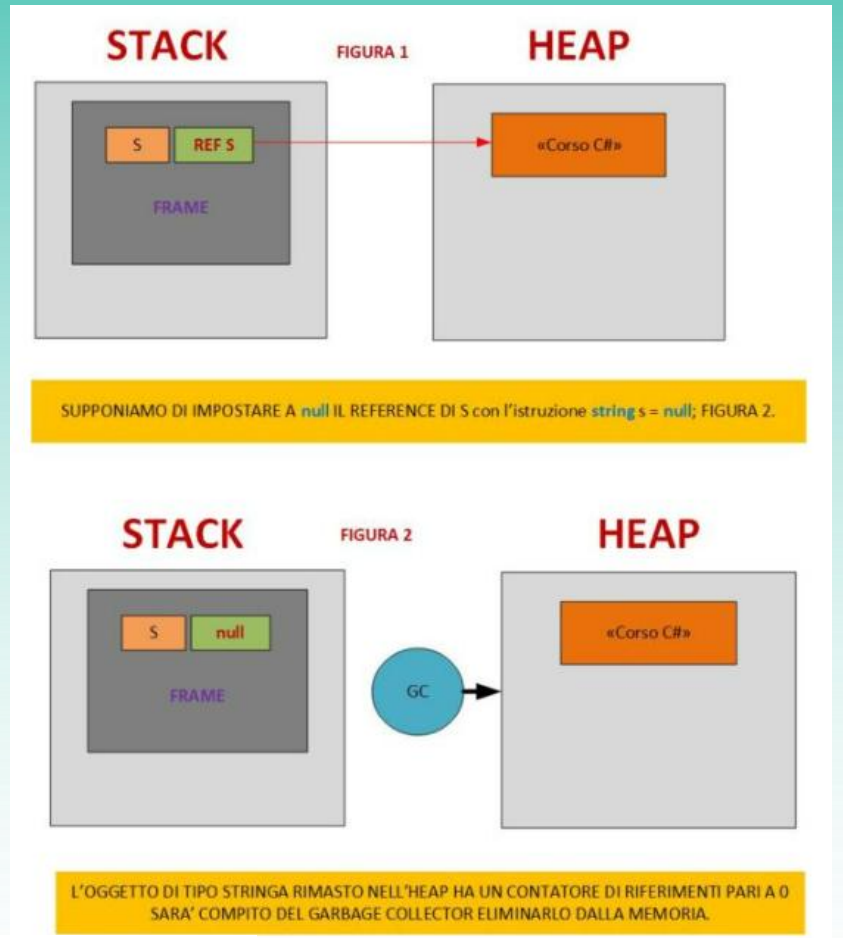
NEL PASSAGGIO DI PARAMETRI PER VALORE QUANDO FIRST METHOD CHIAMA SECOND METHOD PASSANDO L'INTERO 120, ANCHE SE Y MODIFICA QUESTO VALORE IL VALORE DI X AL RITORNO RIMANE INTATTO.



CON IL PASSAGGIO DI PARAMETRI PER RIFERIMENTO QUELLO CHE VIENE PASSATO DAL CHIAMANTE E' UN REFERENCE QUINDI SE NEL METODO CHIAMATO VIENE MODIFICATA LA VARIABILE Y QUESTO SI RIPERCUOTE ANCHE SU X CHE AL RITORNO DAL METODO VEDE MODIFICATO IL SUO VALORE.

Esercitazione4 – Tipi di dato e classi

La keyword null rimuove il riferimento dallo stack che conteneva la locazione di memoria dell'oggetto puntato sull'heap. Se l'oggetto rimane orfano sull'heap, cioè non possiede più riferimenti attivi esiste un meccanismo in C# chiamato garbage collector che esegue una scansione e una pulizia di tutti gli oggetti rimasti orfani nello heap. Se non ci fosse questo meccanismo che è automatico ben presto si arriverebbe ad un memory leak. Vediamo di chiarire il concetto con una immagine.



Esercitazione4 – Tipi di dato e classi

Classe

Una classe è un tipo di riferimento che è possibile derivare direttamente da un'altra classe e che viene derivato in modo implicito da System.Object.

La classe definisce le operazioni che un oggetto (un'istanza della classe) può eseguire (metodi) e i dati che l'oggetto contiene (campi).

Una classe può implementare un numero qualsiasi di interfacce, ma può ereditare solo da una classe di base, oltre che da System.Object, da cui tutte le classi ereditano in modo implicito.

Tutte le classi devono avere almeno un costruttore, per l'inizializzazione di nuove istanze della classe. Se non si definisce in modo esplicito un costruttore, la maggior parte dei compilatori specifica automaticamente un costruttore senza parametri.

Esercitazione4 – Cicli

In alcune situazioni c'è la necessità di ricorrere a dei cicli o istruzioni di iterazione. In C#, questi possono essere costruiti grazie ai cicli: while, for e foreach. Le istruzioni di iterazione possono essere utilizzate per creare cicli per fare in modo che le istruzioni incorporate siano eseguite un determinato numero di volte, in base ai criteri di terminazione del ciclo.

La sintassi di un ciclo while è la seguente:

```
while (condizione booleana)
{
    //ISTRUZIONI
}
```

La condizione booleana è una normale espressione che restituisce true o false. È importante notare che se tale condizione è false prima di iniziare il ciclo (la condizione viene verificata prima) allora il ciclo while non verrà mai eseguito. While itera il blocco di codice racchiuso tra le parentesi graffe fintantoché la condizione è true.

Esercitazione4 – Rami e Cicli

La sintassi del ciclo do-while è la seguente:

```
do

{

    //ISTRUZIONI

} while (condizione booleana)
```

La variante rispetto al ciclo while è che il loop almeno una volta viene eseguito in quanto la condizione booleana è verificata alla fine del blocco di codice.

Esercitazione4 – Rami e Cicli

La sintassi del ciclo for è la seguente:

```
for (condizione iniziale; condizione di test; istruzione di fine loop)

{

        //ISTRUZIONI

}
```

Come vedi il ciclo for è formato nell'intestazione da tre parti, separate da un punto e virgola. La condizione iniziale in cui generalmente viene dichiarata e inizializzata una variabile di controllo, la condizione di test rappresentata da una condizione booleana e l'istruzione di fine loop solitamente rappresentata dall'incremento o decremento della variabile di controllo.

Esercitazione4 – Rami e Cicli

L'ultimo ciclo da esaminare è il foreach. Questo tipo di ciclo serve ad iterare oggetti. Esegue un'istruzione o un blocco di istruzioni per ogni elemento di una istanza di un tipo. Ad esempio:

```
string myVar = "pippo";

foreach(char c in myVar)

{

    Console.WriteLine(c);

}
```

Stamperà tutti i singoli caratteri che si trovano nella stringa pippo.

Mettiamo in pratica Array e Liste E Cicli

Creare una nuova applicazione a console che:

- definisca un array di stringhe, lo valorizzi, e conti quante volte un valore specifico compare nell'array (inserire una stampa a console di un esempio);
- definisca una matrice quadrata di numeri interi, lo valorizzi, e calcoli la somma degli elementi presenti sulla diagonale principale (inserire una stampa a console di un esempio);
- definisca una lista di stringhe, lo valorizzi, e utilizzi un ciclo 'foreach' per cercare una parola specifica nella lista e stampare se la parola della ricerca è presente (inserire una stampa a console di un esempio);
- definisca una lista di numeri interi, lo valorizzi, e utilizzi un ciclo 'foreach' per calcolare la media di tutti gli elementi della lista (inserire una stampa a console di un esempio).

Esercitazione5 – Istruzioni di primo livello

Programmi senza metodo Main

Non è necessario includere in modo esplicito un metodo Main in un progetto di applicazione console.

È invece possibile usare la funzionalità *istruzioni di primo livello* per ridurre al minimo il codice da scrivere.

In questo caso, il compilatore genera una classe Program.

- Un progetto può avere un solo file con istruzioni di primo livello;
- Se si includono direttive using devono venire prima nel file;
- Le istruzioni di primo livello sono in modo implicito nello spazio dei nomi globale;
- Un file con istruzioni di primo livello può contenere anche spazi dei nomi e definizioni dei tipi, ma devono venire dopo le istruzioni di primo livello;
- Per restituire un valore al termine dell'applicazione, usare l'istruzione return come in un metodo Main che restituisce un valore.

Esercitazione6 – Object Oriented

Un linguaggio di programmazione è definito *ad oggetti* quando, usando la sintassi nativa del linguaggio, permette di implementare:

- incapsulamento;
- ereditarietà;
- polimorfismo.

Esercitazione6 – Incapsulamento

Per alterare lo stato interno dell'oggetto, è necessario invocarne i metodi pubblici, ed è questo lo scopo principale dell'incapsulamento.

Infatti, se gestito opportunamente, esso permette di vedere l'oggetto come una *black-box*, cioè una "scatola nera".

Il punto è dare delle funzionalità agli utenti nascondendo i dettagli legati alla loro implementazione.

Esercitazione6 – Ereditarietà

L'ereditarietà consente di creare nuove classi che riutilizzano, estendono e modificano il comportamento definito in altre classi.

La classe i cui membri vengono ereditati è denominata *classe base*, mentre la classe che eredita i membri è denominata *classe derivata*.

Una classe derivata può avere una sola classe di base diretta.

L'ereditarietà, tuttavia, è transitiva. Se ClassC eredita da ClassB e ClassB eredita da ClassA allora ClassC eredita da ClassA.

Esercitazione6 – Polimorfismo

Polimorfismo è una parola che deriva dal greco e significa "multiforme".

Il polimorfismo presenta due aspetti distinti:

- In fase di esecuzione, oggetti di una classe derivata possono essere trattati come oggetti di una classe base.
- Le classi base possono definire e implementare metodi *virtuali* e le classi derivate possono eseguirne l'override, ovvero ne forniscono una propria definizione e implementazione. Durante la fase di esecuzione, quando il codice client chiama il metodo, CLR cerca il tipo in fase di esecuzione dell'oggetto e richiama quell'override del metodo virtuale.

Mettiamo in pratica

Creare una nuova applicazione a console che contenga:

- una classe Persona dotata dei campi:
  1. AnnoDiNascita,
  2. Nome,
  3. Cognome
- una classe Studente come estensione di Persona; i campi aggiuntivi di Studente rispetto a Persona sono:
  1. Matricola,
  2. Università
- la classe Docente che estende Persona; i campi aggiuntivi di Docente rispetto a Persona sono:
  1. Materia
  2. Salario

Ogni classe dovrà avere i metodi per leggere e valorizzare i campi e dovranno essere dotate del metodo toString().

Nel Program.cs creare un oggetto per ogni tipo di classe in cui vengano valorizzati tutti i dati e stampare i dati delle persone inserite.

Infine modificare il codice fiscale dello Studente e stampare a video solo il codice fiscale modificato.

Web API

API

Il termine API, acronimo di Application Programming Interface (interfaccia di programmazione delle applicazioni), indica un insieme di definizioni e protocolli per la creazione e l'integrazione di applicazioni software.

Talvolta vengono concepite come una forma di contratto, con una documentazione che rappresenta un accordo tra le parti: se la parte A invia una richiesta remota strutturata in un determinato modo, il software della parte B risponderà in un altro modo determinato.

Le API sono state introdotte agli albori dell'era informatica, molto prima della comparsa dei personal computer. A quell'epoca, un'API era in genere utilizzata come libreria per un sistema operativo. Era a livello locale rispetto al sistema in cui operava, sebbene a volte passasse messaggi tra mainframe. Dopo circa 30 anni, le API sono emerse dai loro ambienti locali. Nei primi anni 2000 si trasformarono in un'importante tecnologia per l'integrazione remota dei dati.

Le API remote sono concepite per interagire attraverso una rete di comunicazione. Si chiamano API remote perché le risorse gestite da tali API si trovano all'esterno del computer che invia la richiesta. Poiché il canale di comunicazione maggiormente utilizzato è Internet, quasi tutte le API vengono progettate in base agli standard web.

Queste ultime utilizzano in genere l'HTTP per richiedere messaggi e fornire una definizione della struttura dei messaggi di risposta. I messaggi di risposta assumono in genere la forma di file XML o JSON, due formati diffusi perché presentano i dati in modo da consentire alle altre app di gestirli facilmente.

Web API (Web service vs Web API)

L'utilizzo del Web come piattaforma di programmazione è un'idea che si è diffusa intorno al 2000 con l'introduzione dei Web service, cioè di un meccanismo per l'interoperabilità tra applicazioni basato sulle tecnologie e gli standard definiti per il Web.

I Web service consentono di far interagire due applicazioni indipendentemente dal sistema operativo su cui girano e dal linguaggio di programmazione utilizzato. Tecnicamente il meccanismo di interazione è basato sul protocollo SOAP, su un insieme di regole che definiscono la struttura dei messaggi scambiati dalle applicazioni e tutta una serie di accorgimenti per garantire sicurezza, affidabilità, corretta gestione degli eventi, ecc. L'approccio è sostanzialmente basato su una definizione di funzione richiamabile via Web.

Un approccio alternativo ai Web service basati su SOAP, che è stato proposto negli stessi anni ma è stato riscoperto soltanto negli ultimi tempi, è l'approccio noto come REST (Representational State Transfer) e basato su una serie di principi che un'applicazione deve rispettare. In sintesi, nella visione RESTful un Web service non definisce una funzione richiamabile da remoto ma mette a disposizone delle risorse su cui è possibile effettuare le classiche operazioni CRUD sfruttando i metodi del protocollo HTTP.

Due visioni diverse del Web come piattaforma di elaborazione che sempre più spesso vengono distinte parlando semplicemente di Web service quando si fa riferimento a quelli basati si SOAP e di Web API quando ci ispira al modello REST.

Web API (SOAP vs REST)

Vista la continua diffusione delle API web, è stato sviluppato un protocollo specifico con lo scopo di standardizzare lo scambio delle informazioni, denominato Simple Object Access Protocol, o protocollo SOAP. Le API progettate con il protocollo SOAP usano il linguaggio XML come formato del messaggio e ricevono le richieste tramite HTTP o SMTP. Il protocollo SOAP facilita la condivisione delle informazioni per le app eseguite su ambienti diversi o scritte in linguaggi differenti.

È stata introdotta anche la specifica Representational State Transfer (REST). Le API web che rispettano i vincoli architetturali REST vengono definite API RESTful.

La differenza tra REST e SOAP è sostanziale: il SOAP è un protocollo, mentre il REST è un tipo di architettura e ciò implica l'assenza di uno standard ufficiale per le API web RESTful.

Web API (API REST)

Le API sono definibili RESTful se rispettano i sei vincoli di un sistema RESTful:

- Architettura client-server: l'architettura REST è costituita da client, server e risorse e gestisce le richieste tramite il protocollo HTTP.
- Stateless: nessun contenuto client è archiviato nel server tra le richieste. Le informazioni relative allo stato della sessione sono invece contenute nel client.
- Supporto cache: il caching può eliminare la necessità di alcune interazioni client-server.
- Sistema a livelli: le interazioni client-server possono essere mediate da livelli aggiuntivi, i quali possono offrire altre funzionalità, come bilanciamento del carico, condivisione della cache o sicurezza.
- Codice on demand (opzionale): i server possono ampliare la funzionalità di un client trasferendo del codice eseguibile.
- Interfaccia uniforme: è il vincolo principale per la progettazione di API RESTful e prevede 4 aspetti:
    1. Identificazione delle risorse nelle richieste: le risorse vengono identificate nelle richieste e vengono distinte dalle rappresentazioni restituite al client.
    2. Manipolazione delle risorse tramite le rappresentazioni: i client ricevono file che rappresentano le risorse e che devono contenere le informazioni necessarie per consentirne la modifica o l'eliminazione.
    3. Messaggi autodescrittivi: ogni messaggio restituito a un client contiene le informazioni necessarie per descrivere come il client deve elaborare l'informazione.
    4. Ipermedia come motore dello stato dell'applicazione: accedendo alla risorsa, il client REST deve poter individuare, attraverso hyperlink, tutte le altre azioni disponibili al momento.

Benché sembrino numerosi, questi vincoli sono molto più semplici rispetto a un protocollo prescritto e, proprio per questo, le API RESTful sono molto più diffuse dei metodi SOAP.

Negli ultimi anni, la specifica OpenAPI si è imposta come standard comune per la definizione delle API REST, poiché consente agli sviluppatori di realizzare interfacce API REST indipendenti dal linguaggio e facilmente comprensibili agli utenti.

Web API

API Web di base ASP.NET

ASP.NET Core è un framework open source, multipiattaforma e ad alte prestazioni per la creazione di app moderne, abilitate al cloud e connesse a Internet.

Con ASP.NET Core si può:

- Creare app e servizi web, app per backend per dispositivi mobili
- Utilizzare strumenti di sviluppo su Windows, macOS e Linux
- Distribuire sul cloud o in locale
- Eseguire l'app su .NET

Web API (openAPI)

Supporto openAPI nelle app per le API principali di ASP.NET

La specifica OpenAPI è uno standard indipendente dal linguaggio di programmazione per la documentazione delle API HTTP. Questo standard è supportato in API minime tramite una combinazione di API predefinite e librerie open source. L'integrazione OpenAPI in un'applicazione presenta tre aspetti chiave:

- Generazione di informazioni sugli endpoint nell'app.
- Raccolta delle informazioni in un formato corrispondente allo schema OpenAPI.
- Esposizione dello schema OpenAPI generato tramite un'interfaccia utente visiva o un file serializzato.

Esercitazione7 – Web API

- Nel terminale di Visual Studio Code spostarsi nella directory in cui si vuole creare il nuovo progetto;
- Creare un nuovo progetto .NET 8 di tipo web API denominato Esercitazione7
  - ~~`dotnet new webapi -n 7-demoApi8 -f net8.0`~~ **il trattino è sbagliato!!**

```
PS C:\Dev\CorsoITS\MyExercises> dotnet new webapi -n 7-demoApi8 -f net8.0
>>
Errore: opzioni non valide:
-n
   '-n' non è un'opzione valida
7-demoApi8
   '7-demoApi8' non è un'opzione valida
-f
   '-f' non è un'opzione valida
net8.0
   'net8.0' non è un'opzione valida

Per altre informazioni, eseguire:
   dotnet new webapi -h

Per informazioni dettagliate sul codice di uscita, vedere https://aka.ms/templating-exit-codes#127
PS C:\Dev\CorsoITS\MyExercises> dotnet new webapi -n 7-demoApi8 -f net8.0
>>
Creazione del modello "API Web ASP.NET Core" completata.
```

  - `dotnet new webapi -n 7-demoApi8 -f net8.0`

Esercitazione7 – Web API

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
var app = builder.Build();
if (app.Environment.IsDevelopment()){
    app.UseSwagger();
    app.UseSwaggerUI();}
app.UseHttpsRedirection();
var summaries = new[]{"Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot",
"Sweltering", "Scorching"};
app.MapGet("/weatherforecast", () =>{
    var forecast =  Enumerable.Range(1, 5).Select(index =>
        new WeatherForecast(
            DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
            Random.Shared.Next(-20, 55),
            summaries[Random.Shared.Next(summaries.Length)])).ToArray();
    return forecast;})
.WithName("GetWeatherForecast")
.WithOpenApi();
```

Program.cs

Le API minime offrono il supporto predefinito per la generazione di informazioni sugli endpoint in un'app tramite il pacchetto Microsoft.AspNetCore.OpenApi.

- AddEndpointsApiExplorer : configura l'app per l'uso di Esplora API per individuare e descrivere gli endpoint con annotazioni predefinite.
- WithOpenApi esegue l'override delle annotazioni predefinite generate da Esplora API con quelle generate dal pacchetto Microsoft.AspNetCore.OpenApi.
- UseSwagger aggiunge il middleware Swagger.
- 'UseSwaggerUI' abilita una versione incorporata dello strumento dell'interfaccia utente di Swagger.
- WithNameIEndpointNameMetadata: nell'endpoint viene usato per la generazione di collegamenti e viene considerato come l'ID operazione nella specifica OpenAPI dell'endpoint specificato.

Esercitazione7 – Web API

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi"
Version="8.0.8" />
    <PackageReference Include="Swashbuckle.AspNetCore"
Version="6.4.0" />
  </ItemGroup>

</Project>
```

Esercitazione7.csproj
Microsoft.AspNetCore.OpenApi viene aggiunto come PackageReference a un file di progetto.